

Enterprise MCP Gateway - Principal Architecture Audit & Technical Baseline

Document Type: Architecture Discovery Baseline **Audit Date:** September 2026
Repository: enterprise-mcp-gateway-core **Auditor Role:** Principal AI Architect &
Lead Technical Auditor

Executive Summary

The Enterprise MCP Gateway is a **production-deployed, multi-tenant MCP server** that exposes 39 tools across 7 backend systems (Jira, Business Rule Engine/Reference Data Service, Neo4j, Teradata, DocumentDB, Enterprise SQL Server, and an Enterprise LLM Orchestrator) with enterprise-grade security, observability, and governance. It is not a basic proxy. It implements dual-layer authentication, AD-group-based RBAC, seven composable security guardrails, two-layer rate limiting, KMS envelope encryption, three parallel audit pipelines, and a sidecar-supervised multi-backend architecture -- all backed by DocumentDB as the dynamic configuration plane.

Section 1: Identity & Token Lifecycle

1A. JWT Validation Engine

The gateway accepts JWTs from **three distinct issuers** through a single configurable validator:

- **Gateway-signed tokens** (Kiro IDE): HS256 signed with `JWT_SECRET`, containing embedded RBAC claims (`allowed_tools`, `projects`, `rate_limit`, `role`, `groups`)
- **Upstream service JWTs** (AI Agent): HS512/HS256/RS256 signed with the enterprise OAuth client secret, carrying identity claims from the Enterprise IdP
- **External IdP tokens:** Configurable claim field mapping (`JWT_CLAIM_USER_ID`, `JWT_CLAIM_GROUPS`, etc.) makes the validator IdP-agnostic

The `JWTClaims` dataclass provides a normalized view regardless of source: `user_id`,

`username`, `email`, `groups`, `raw_payload`. The `_extract_claim()` function accepts comma-separated candidate field names and tries each in order, supporting IdPs that use different claim schemas without code changes.

`validate_jwt_token()` never raises -- it returns `None` on any failure (expired, bad signature, missing secret, import error), making it safe to call speculatively. Algorithm negotiation supports HS512, HS256, and RS256 simultaneously.

1B. Three-Mode Authentication Dispatch

The `AuthMiddleware` implements three authentication modes in a single request pipeline:

Mode	Trigger	Identity Source	Scope Source
API Key + JWT	<code>X-MCP-API-Key</code> header present	JWT <code>Authorization: Bearer</code> (overrides headers)	Consumer registry (<code>mcp_consumers</code>)
Kiro RBAC	No API key, app name starts with <code>kiro-</code> , <code>KIRO_RBAC_ENABLED=true</code>	Gateway-signed JWT in <code>X-MCP-Auth-Token</code>	JWT claims (embedded from RBAC role at issuance)
Kiro Legacy	No API key, app name starts with <code>kiro-</code> , <code>KIRO_RBAC_ENABLED=false</code>	Request headers (trusted)	Wildcard (all tools, all projects)

Each mode produces a `CallerIdentity` stored in both `request.state.caller` and a `contextvars.ContextVar` for async-safe downstream access by audit loggers, rate limiters, and tool handlers.

1C. Token Generation & RBAC Resolution

The Kiro token generation endpoint implements the full token lifecycle:

1. Extract user identity from the Enterprise IdP OAuth cookie (three cookie names tried, plus Bearer header fallback)
2. Resolve AD groups: JWT `groups` claim first, then server-side lookup from the enterprise user profile database, then permissions collection fallback
3. RBAC resolution (three-tier priority):
4. Profile override (`mcp_kiro_profiles`): `is_active=false` = DENIED,

- `is_active=true` = custom scope
- 5. AD group match (`mcp_rbac_roles`): case-insensitive match against 5 seeded roles
 - no per-user provisioning needed
- 6. Neither = DENIED with portal link
- 7. Sign JWT with `jose.jwt.encode(HS256)`, `embedding` `allowed_tools`, `projects`, `rate_limit`, `role`, `groups`, `token_type: "kiro-access"`, `gateway_env`
- 8. Audit every attempt (success/failure) to the access audit collection

1D. Credential Resolution Pipeline

For AI-agent consumers, the middleware resolves Jira PAT and LLM orchestrator keys through a three-tier fallback:

- 1. Request header (`X-MCP-Jira-PAT`) -- highest priority, set by the calling agent
- 2. In-process DocumentDB lookup + AWS KMS decryption
- 3. HTTP decrypt endpoint fallback with 5s timeout

Credential source tracking (`_jira_pat_source_var`) flows through to audit logs and guardrail decisions.

Section 2: Enterprise Security & Fine-Grained Authorization

2A. Seven Composable Guardrails

The `enforce_guardrails()` function runs a pipeline of seven checks in sequence. Every check returns a structured error dict or `None` (pass). All guardrail settings are **DocumentDB-backed with env-var fallback** (30s cache TTL):

#	Check	What It Does
0	Tool Active	Registry lookup; disabled tools return <code>tool_disabled</code> error
1	Tool Authorization	Validates requested tool against consumer's <code>allowed_tools</code> whitelist
2	User Write Authorization	AD-group gate for write operations on <code>ai-agent</code> consumers. Resolution: JWT groups then enterprise user profile roles
3	Write Credential Attribution	Requires personal Jira PAT for writes (<code>source = request_header</code> or <code>profile</code>), preventing changes under shared service account

4	JQL Input Validation	Blocks SQL injection patterns (;DROP, --, /* */), hex/unicode escapes), 2000-char limit
5	SQL Read-Only Validation	Teradata: only single SELECT/WITH, blocks all DDL/DML, 5000-char limit
6	Project Scope Enforcement	Validates issue keys against PROJECT-123 format, checks consumer's project whitelist, parses JQL for project references
7	Write Rate Limiting	Per-consumer sliding window (default 5 writes per 600s)

2B. RBAC Role Architecture

Five AD-group-to-role mappings stored in `mcp_rbac_roles` collection:

AD Group	Role	Tools	Projects	Rate Limit
Platform-Admin	admin	All (*)	All (*)	30/min
Platform-Write-Dev	developer	All (*)	All (*)	15/min
Platform-Write	developer	All (*)	All (*)	15/min
Platform-Standard-Dev	reader	27 specific read-only tools	6 projects	10/min
Platform-Supervisor-Dev	supervisor	All (*)	All (*)	20/min

RBAC roles are cached for 5 minutes with cache invalidation on CRUD operations.

2C. Consumer Registry (Multi-Source)

The consumer registry loads from multiple sources with priority:

1. **DocumentDB** (`mcp_consumers`) -- production source of truth
2. **AWS Secrets Manager** -- legacy support
3. **Local file** (`config/consumers.json`) -- development fallback

Each `ConsumerProfile` carries: `api_key`, `consumer name`, `type` (developer/ai-agent/rest-api/kiro-ide), `allowed_tools`, `rate_limit`, `projects`, `is_active`.

2D. AWS KMS Envelope Encryption

The encryption module implements enterprise-grade token encryption:

- **KMS envelope encryption (AES-256-GCM):** KMS generates a Data Encryption

Key per operation, DEK encrypts the plaintext, KMS encrypts the DEK with the CMK. Only `encrypted_token + encrypted_DEK + nonce` are stored -- no plaintext anywhere.

- **Fernet fallback** for local development (auto-generated key with restart warning)
- **CloudTrail integration:** Every KMS decrypt call is logged (audit by AWS)
- **Key rotation:** Via KMS without re-encrypting stored data

2E. Business Rule Engine Role Authorization

The BRE role module provides Business Rule Engine-specific access control. It validates the user's personnel identifier against BRE role assignments, with a human-readable access summary for the access check tool.

2F. Jira OAuth PAT Automation

The OAuth client implements the full OAuth 1.0a RSA-SHA1 flow for automated Jira PAT creation:

- Request token, access token exchange, PAT creation, revocation, rotation
- RSA-SHA1 signature with PEM private key from Secrets Manager
- PAT lifecycle management (create, list, rotate, revoke)

Section 3: Client Access & Proxy Governance

3A. Two-Layer Rate Limiting

The rate limiter implements a sliding-window algorithm with two enforcement layers:

Layer	Scope	Source	Purpose
Layer 1 (inner)	Per-consumer per-tool-group	<code>ConsumerProfile.rate_limit</code> from RBAC JWT claims	Individual user throttling (e.g., 15/ min for developers)
Layer 2 (outer)	Global per-tool-group	DocumentDB <code>mcp_tool_groups.rate_limit</code> (default 59 -- one under Jira's 60 burst limit)	Backend protection across all consumers

Bypass logic: When a caller provides their own Jira PAT or LLM orchestrator key (source != `server_config`), rate limiting is skipped entirely -- they consume their own API quota.

The `@rate_limited` decorator wraps every tool function and returns structured error with `retry_after_seconds` on breach.

3B. Node.js Auth Proxy (Kiro IDE)

A **zero-dependency** Node.js stdio-to-HTTPS proxy bridges Kiro IDE to the gateway:

- JWT token management with expiry checking (5-min buffer)
- Browser-based auto-authentication (opens Kiro Access page when token is missing/expired)
- `fs.watch` + polling fallback (2s interval, 5-min timeout) for credential file detection
- Automatic re-auth on 401/403 with single retry
- SSE response parsing for MCP Streamable HTTP
- Forwards `X-MCP-Jira-PAT` and LLM orchestrator keys from env vars

3C. Dynamic Tool Registry

The tool registry is a thread-safe, DocumentDB-backed metadata registry:

- Configurable TTL (default 300s)
- Stale-cache-on-failure: if DocumentDB is unavailable, serves the last known good state
- `is_tool_active()` enables dynamic tool disable without code deploy
- `get_write_tools()` provides the write-tool set for guardrail decisions

3D. Response Caching (Redis-Backed)

The distributed response cache provides:

- Namespace-aware (jira, bre, teradata) with per-namespace TTL
- Jira TTL override from DocumentDB
- Targeted invalidation on write operations
- Shared stamps for cross-pod cache coordination
- Health check with connection status
- Automatic unavailable marking with degraded-mode operation

The `@cached` decorator auto-detects namespace from tool name, only caches successful

responses, and normalizes cache keys from function signatures.

3E. Reference Data Picklist Cache (Long-Lived, Scheduled Refresh)

A specialized cache for reference data service picklists:

- Disk-backed persistence (pickle serialization)
- Scheduled daily refresh at 06:00 America/Chicago timezone
- Redis-based cluster-wide invalidation stamps
- DocumentDB-configurable TTL and refresh schedule
- Staleness detection combining TTL + boundary-crossing + global invalidation

Section 4: Observability & Compliance

4A. Three Parallel Audit Pipelines

Pipeline	Collection	Decorator	Scope
Jira Audit	MCPJiraAudit	@jira_audited	All 12 Jira tools + guardrail blocks
Generic Audit	MCPAuditLog	@audit_log	BRE, Neo4j, Teradata tools
Kiro Access Audit	MCPKiroAudit	_audit_token_generation()	Token generation success/failure

Jira Audit schema: timestamp, tool, backend (pat/llm-orchestrator/basic), dashboard, caller, caller_identity (user_id, username, email, groups, consumer, credential_fingerprint), status (ok/error/rate_limited/timeout/exception/mock), duration_ms, request (sanitized), response_summary (issue_keys, counts), error (structured), cache_hit, rate_limit, metadata (server_version, environment).

Unified Dashboard: Queries BOTH collections, normalizes schemas, merge-sorts by timestamp, provides aggregation endpoints for stats and usage metrics.

4B. Log Sanitization & Credential Fingerprinting

- `redact_for_log()`: Recursive redaction of sensitive fields (authorization, api_key, token, pat, password, secret, cookie, credential, session). Shows first 4 + last 4 chars. Bounded: max 500 chars/string, max 50 items/collection.

- `secret_fingerprint()`: SHA-256 first 12 hex chars -- non-reversible correlation ID for tracking credential usage across audit entries without exposing the secret.

4C. In-Memory Metrics

Thread-safe collector (max 2000 entries, FIFO eviction). Tracks tool, group, status, duration_ms, cache_hit. Provides period aggregation (by_group, by_status, avg_duration, cache_hit_rate). Exposed via `get_server_metrics` MCP tool.

Section 5: System Resilience & Error Handling

5A. Fail-Open vs. Fail-Closed Decisions

Component	Failure Mode	Rationale
Audit logger	Fail-open: <code>_audit_disabled</code> flag prevents tool calls from blocking on DB failure	Tool availability > audit completeness
Tool registry	Fail-open: Serves stale cache if DocumentDB is unavailable	Tool availability > metadata freshness
Rate limiter	Fail-open: Returns cached/empty limits if DocumentDB is unavailable	Tool availability > precise limiting
Consumer registry	Fail-open: Falls through DocumentDB then Secrets Manager then local file then defaults	Graceful degradation through multiple tiers
Redis cache	Fail-open: Cache miss falls through to direct tool execution	Performance > availability
Write authorization	Fail-closed: No configured AD groups = deny all writes	Security > availability for writes
Kiro RBAC	Fail-closed: Missing/invalid token = 401	Security > convenience
Wildcard expansion	Fail-closed: DB unavailable during expansion = 503 (not store " <code>*</code> ")	Security > data integrity
BRE role lookup	Fail-closed: Missing profile = deny	Security > convenience

5B. Structured Error Taxonomy

Every error returns a predictable JSON shape:

Error Type	Key Fields	HTTP Status
rate_limited	group, retry_after_seconds, limit_type (consumer/global)	200 (in MCP tool response)
write_rate_limited	guardrail, retry_after_seconds	200 (in MCP tool response)
forbidden	guardrail (tool_authorization/project_scope/user_write_auth/write_credential_attribution)	200 (in MCP tool response)
invalid_input	guardrail (input_validation/sql_readonly)	200 (in MCP tool response)
tool_disabled	guardrail (tool_active_check)	200 (in MCP tool response)
maintenance	maintenance_window, retry_after, suggestion	200 (in MCP tool response)
unauthorized / kiro_auth_required / kiro_token_invalid	error, message, portal_url	401
no_kiro_access / profile_inactive	error, message	403
teradata_unavailable / teradata_mcp_error	error, message, tool	200 (in MCP tool response)
service_unavailable	error, message	503

5C. Sidecar Supervision

The container endpoint implements a supervised sidecar pattern for the Teradata MCP server:

- Background loop with exponential backoff (2s initial, doubles, caps at 60s)
- Credential resolution via helper script (derives from reference data service credentials, never echoes password)
- Fail-safe: if URI can't be resolved, gateway stays up; tools return `teradata_unavailable`
- Gateway execs as PID 1 (proper signal handling for Kubernetes)

5D. Teradata MCP Client Resilience

The Teradata MCP sidecar client handles the full MCP JSON-RPC handshake: - Session management with automatic re-handshake on session expiry (400/404) - Thread-safe with `threading.Lock` (serialized calls -- acceptable for read-only low-volume Teradata) - SSE response parsing (handles both `application/json` and `text/event-stream`) - Error text truncation (500 chars max) to prevent large Teradata error payloads from bloating responses - Health probe with bounded timeout (min of configured timeout, 10s)

5E. Maintenance Window System

DocumentDB-backed maintenance windows with timezone handling. Save writes a persistent record; clear writes a "disabled" sentinel (not a delete) so all pods see the authoritative state via 30s cache TTL.

Section 6: Advanced Architectural Highlights

These patterns elevate the gateway from a basic proxy to enterprise-grade:

- 1. DocumentDB as Dynamic Configuration Plane:** Guardrail thresholds, rate limits, tool enable/disable, maintenance windows, write AD groups, Jira backend mode, cache TTL -- all configurable at runtime via DocumentDB with env-var fallback. Changes propagate across pods within 30s cache TTL without deploys.
- 2. Stateless HTTP MCP Transport:** `stateless_http=True` removes the MCP session handshake requirement, enabling plain HTTP consumers (Java RestTemplate, Python httpx) to `POST tools/call` directly without an `initialize` round-trip.
- 3. Credential Source Tracking:** Every credential (Jira PAT, LLM orchestrator key) carries a `source` tag (`request_header`, `profile`, `server_config`) through the entire request lifecycle. Audit logs record the source, guardrails make decisions based on it, and rate limiting bypasses on user-provided credentials.
- 4. Cohort-Based Intelligent Defaulting:** The BRE defaulting rules module provides ML-lite field suggestions for document creation based on historical cohort patterns. Rules are stored in DocumentDB with staleness thresholds (45 days) and support floors (50 rows).
- 5. Cross-Pod Cache Coordination via Redis Stamps:** Shared timestamp values

enable cluster-wide cache invalidation. When one pod flushes a cache, other pods detect the invalidation timestamp and re-fetch on next access.

6. **Decorator Composition Order:** Every tool follows a consistent decorator stack: `@audit_decorator` (outermost) -> `@cached` -> `@rate_limited` -> `@guardrail` -> tool logic. This ensures audit captures rate-limited and guardrail-blocked calls, caching sits above rate limiting, and guardrails are the innermost gate.
 7. **Enterprise LLM Orchestrator Integration:** The LLM orchestrator client implements full MCP-over-JSON-RPC bridging to the enterprise AI gateway. Tool name mapping, environment detection from Jira URL, SSE response parsing, and async/sync dual API.
 8. **Multi-Backend Jira Dispatch:** Each Jira tool dynamically selects its backend at call time: PAT (direct REST with Bearer token), Basic (email + token), or Enterprise LLM Orchestrator (JSON-RPC). Backend selection is DocumentDB-backed with per-dashboard override capability.
 9. **AD Group Resolution Without LDAP:** Instead of requiring direct LDAP access (often blocked in containerized environments), the gateway resolves AD groups from the existing enterprise user profile database -- data already populated by the broader platform.
 10. **Audit-Driven Guardrail Integration:** Guardrail enforcement is embedded within audit decorators. Blocked calls are audited with full context (caller, requested tool, block reason) before the rejection response is returned.
-

Section 7: Comprehensive Inventory Table

Architectural Domain	Discovered Capability / Design Pattern	Impact on Gateway Security/Performance
JWT Validation Engine	IdP-agnostic JWT validation with configurable claim mapping; multi-algorithm support (HS512, HS256, RS256); never-raise design returns None on failure	Enables multi-issuer token acceptance without code changes; safe speculative validation prevents auth crashes
Authentication Middleware	Three-mode auth dispatch (API Key + JWT, Kiro RBAC, Kiro Legacy); per-request <code>ContextVar</code> identity propagation; credential resolution with 3-tier fallback	Single entry point for all auth decisions; async-safe identity flow to downstream audit, rate limiting, and guardrails

Kiro Token Lifecycle	Gateway-signed JWT with embedded RBAC claims; AD group resolution from enterprise user DB (no LDAP); profile override layer; comprehensive audit of every token attempt	Users get scoped tokens without per-user provisioning; server-side AD resolution works in containerized environments
RBAC Role Mapping	CRUD for AD-group-to-MCP-role mappings in DocumentDB; cache invalidation on writes; 5 seeded roles with graduated permissions	AD-group membership auto-grants Kiro access at appropriate scope; no per-user provisioning scales to entire organization
Consumer Registry	Multi-source loading (DocumentDB -> Secrets Manager -> local file -> defaults); tool and project authorization	Graceful multi-tier degradation; per-consumer scoping prevents tool abuse
Security Guardrails Pipeline	7 composable checks: tool auth, write auth, credential attribution, JQL validation, SQL read-only, project scope, write rate limiting; DocumentDB-backed dynamic config	Defense-in-depth: every tool call passes 7 independent security gates; runtime-tunable without deploys
KMS Envelope Encryption	AES-256-GCM with KMS-managed DEK; Fernet fallback for dev; CloudTrail audit of every decrypt	Tokens encrypted at rest in DocumentDB; master key never leaves KMS HSM; key rotation without re-encryption
Two-Layer Rate Limiter	Per-consumer sliding window (RBAC-scoped) + per-tool-group global limit; user-credential bypass; structured retry_after	Prevents individual abuse AND backend overload; user-PAT bypass respects API quota ownership
Jira Audit Trail	DocumentDB + structured logger dual-write; full caller identity with credential fingerprint; guardrail integration audits blocked calls	Complete traceability of every Jira operation; blocked calls audited before rejection
Generic Audit Trail	Schema-aligned with Jira audit; whitelist-based response summarization; fail-open <code>_audit_disabled</code> flag; tool source metadata injection	Consistent audit across all tool groups; audit failures don't block tool execution
Kiro Access Audit	Every token generation attempt logged (success/failure); AD groups captured at issuance; client IP and user agent; aggregation stats API	Full access audit trail for compliance; enables detection of unauthorized access attempts

Log Sanitization	Recursive credential redaction (first4 + last4); SHA-256 fingerprinting for non-reversible correlation; bounded output (500 chars, 50 items)	Prevents credential leakage in logs while maintaining traceability
Redis Response Cache	Namespace-aware distributed caching; targeted invalidation on writes; cross-pod stamp coordination; DocumentDB-configurable TTL	Reduces backend load; cross-pod consistency via Redis stamps
Reference Data Picklist Cache	Disk-backed persistence; scheduled daily refresh; Redis cluster-wide invalidation; DocumentDB-configurable	Reference data available even during backend outages; scheduled refresh keeps data fresh
Node.js Auth Proxy	Zero-dependency stdio-to-HTTPS bridge; JWT expiry checking; browser-based auto-auth; credential file detection; 401 re-auth with single retry; SSE parsing	Fully automated Kiro IDE authentication; distributable as a single file
Tool Registry	Thread-safe DocumentDB-backed registry; stale-cache-on-failure; dynamic enable/disable without deploy	Kill switch for any tool at runtime; guardrails reference registry for tool_active check
Jira Multi-Backend Dispatch	Dynamic backend selection (PAT/Basic/LLM Orchestrator); DocumentDB-configurable per-dashboard; multi-dashboard support	Enables AI Gateway integration without code changes; per-dashboard routing
Enterprise LLM Orchestrator Client	MCP-over-JSON-RPC to enterprise AI gateway; environment detection from Jira URL; SSE response parsing; async/sync dual API	Enterprise AI Gateway compliance; tool name translation layer
BRE Workflow Engine	15 tools with extensive field validation, picklist cross-checking; date/boolean normalization; role-based authorization; server-side inter-system attachment transfer	Data integrity for business rule documents; prevents invalid submissions
BRE Intelligent Defaulting	Cohort-based field suggestions from historical patterns; staleness threshold (45 days); support floor (50 rows); two-tier specificity matching	ML-lite assistance for document creation; reduces user error

Neo4j Knowledge Graph	Multi-instance support (live + experimentation); regex-based Cypher validation (read-only); schema introspection with TTL cache; auto-limit injection	Safe read-only access; prevents write operations at query level
Teradata MCP Sidecar	Supervised sidecar with exponential backoff restart; MCP JSON-RPC handshake; auto-re-handshake; read-only enforced at 4 layers (DB account, sidecar profile, gateway SQL guard, row cap)	Four-layer read-only enforcement; sidecar supervision ensures recovery
OAuth PAT Automation	Full OAuth 1.0a RSA-SHA1 flow; PAT lifecycle management (create/list/rotate/revoke); PEM key from Secrets Manager	Automated credential rotation; eliminates manual PAT management
Stateless HTTP Transport	Removes MCP session handshake requirement; plain HTTP POST consumers supported	Enables Java/Python service consumers without MCP SDK
Feature Flags	6 CI/CD-variable-driven toggles; graceful degradation when disabled	Safe incremental rollout; easy rollback without deploy

Section 8: DocumentDB Collections Map

Collection	Purpose
mcp_consumers	Consumer API key registry
mcp_rbac_roles	AD-group-to-MCP-role mappings
mcp_kiro_profiles	Per-user profile overrides
mcp_tool_registry	Tool metadata (name, group, category, is_active)
mcp_tool_groups	Tool group config (rate_limit, display_name)
mcp_jira_projects	Jira project scope definitions
mcp_gateway_config	Dynamic configuration (guardrails, backend mode, write AD groups, cache TTL)
mcp_bre_defaulting_rules	BRE cohort-based field suggestion rules
MCPJiraAudit	Jira tool audit trail

MCPAuditLog	Generic tool audit trail (BRE, Neo4j, Teradata)
MCPKiroAudit	Kiro access/token generation audit

Section 9: Tool Inventory (39 Tools)

#	Tool Name	Group	Category	Backend
1	get_jira_issues	jira	read	PAT/Basic/LLM Orchestrator
2	jira_get_issue_details	jira	read	PAT/Basic/LLM Orchestrator
3	jira_create_issue	jira	write	PAT/Basic/LLM Orchestrator
4	jira_assign_issue	jira	write	PAT/Basic/LLM Orchestrator
5	jira_transition_issue	jira	write	PAT/Basic/LLM Orchestrator
6	jira_add_comment	jira	write	PAT/Basic/LLM Orchestrator
7	jira_attach_file	jira	write	PAT only
8	jira_get_attachments	jira	read	PAT/Basic/LLM Orchestrator
9	jira_get_comments	jira	read	PAT/Basic/LLM Orchestrator
10	jira_get_sprint_issues	jira	read	PAT/Basic/LLM Orchestrator
11	jira_search_users	jira	read	PAT/Basic/LLM Orchestrator
12	jira_list_dashboards	jira	read	PAT/Basic/LLM Orchestrator
13	bre_check_access	bre	read	Reference Data Service API
14	bre_get_intake_details	bre	read	Enterprise SQL Server

15	bre_get_details	bre	read	Reference Data Service API
16	bre_search	bre	read	Teradata
17	bre_get_intake_mapping	bre	read	Reference Data Service API
18	bre_get_approval_chain	bre	read	Reference Data Service API
19	bre_get_suggested_default	bre	read	DocumentDB
20	bre_get_picklists	bre	read	Reference Data Service API (cached)
21	bre_refresh_picklists	bre	write	Reference Data Service API + Redis
22	bre_validate_draft	bre	read	Reference Data Service API
23	bre_create	bre	write	Reference Data Service API
24	bre_update_status	bre	write	Reference Data Service API
25	bre_link_intake	bre	write	Reference Data Service API
26	bre_attach_file	bre	write	Reference Data Service API
27	bre_attach_jira_attachment	bre	write	Jira + Reference Data Service API
28	neo4j_get_schema	neo4j	read	Neo4j Bolt
29	neo4j_validate_query	neo4j	read	Regex (local)
30	neo4j_run_query	neo4j	read	Neo4j Bolt
31	teradata_read_query	teradata	read	MCP Sidecar
32	teradata_list_databases	teradata	read	MCP Sidecar
33	teradata_list_tables	teradata	read	MCP Sidecar
34	teradata_describe_columns	teradata	read	MCP Sidecar

35	teradata_get_column_metadata	teradata	read	MCP Sidecar
36	teradata_get_table_ddl	teradata	read	MCP Sidecar
37	teradata_preview_table	teradata	read	MCP Sidecar
38	get_server_metrics	built-in	read	In-memory
39	health_check	built-in	read	Multi-backend probe

Section 10: Key Statistics

Metric	Value
Total MCP tools	39 (12 Jira + 15 BRE + 3 Neo4j + 7 Teradata + 2 built-in)
Backend systems	7 (Jira REST, Enterprise LLM Orchestrator, Reference Data Service API, Enterprise SQL Server, Teradata, Neo4j, DocumentDB)
Dashboard API endpoints	~30+ across 15 route modules
DocumentDB collections	11
Security guardrails	7 composable checks
Audit pipelines	3 parallel (Jira, Generic, Kiro Access)
Authentication modes	3 (API Key + JWT, Kiro RBAC, Kiro Legacy)
Rate limiting layers	2 (per-consumer + per-tool-group)
Cache layers	3 (Redis response cache, reference data picklist disk cache, in-memory registry cache)
Encryption	AES-256-GCM with KMS envelope encryption
Deployment environments	4 (sandbox, dev, UAT, prod)
Feature flags	6 (JWT_AUTH, KIRO_RBAC, NEO4J, TERADATA_MCP, CACHE, USER_WRITE_AUTH)

This baseline establishes the Enterprise MCP Gateway as a multi-tenant, multi-backend,

enterprise-grade API gateway with defense-in-depth security, comprehensive observability, and production-hardened resilience patterns. The combination of DocumentDB-driven dynamic configuration, seven composable guardrails, three audit pipelines, and multi-mode authentication positions this well above a basic MCP proxy -- it is an enterprise gateway platform.